



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ

Лабораторная работа №1

«Основные конструкции языка Python»

ДИСЦИПЛИНА: «Архитектура автоматизированных систем обработки информации и управления»

Выполнил: студенты гр. ИУ5-32Б

(Подпись) Маслов А. О.
(Ф.И.О.)

Проверил:

(Подпись) Гапанюк Ю. Е.
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Москва, 2026 г.

Цель лабораторной работы

Изучение основных конструкций языка Python.

Описание сделанной работы

Разработать программу для решения биквадратного уравнения ($Ax^4 + Bx^2 + C = 0$).

- Программа разрабатывается в виде консольного приложения на языке Python.
- Осуществляется ввод с клавиатуры коэффициентов A, B, C, вычисляется дискриминант и действительные корни уравнения.
- Коэффициенты могут быть заданы в виде параметров командной строки. Если они не заданы, осуществляется ввод с клавиатуры.
- При вводе некорректного значения (например, букв) программа игнорирует его и запрашивает коэффициент повторно до успешного ввода.

Процедурный подход

```
import sys
import math

def get_coef(index, prompt, nonzero=False):
    if index < len(sys.argv):
        value = sys.argv[index]
    else:
        value = None
    while True:
        if value is None:
            value = input(prompt)
        try:
            coef = float(value)
            if nonzero and coef == 0:
                print("Коэффициент A не может быть равен 0")
                value = None
                continue
            return coef
        except ValueError:
            print("Некорректное значение, введите действительное число.")
            value = None

def get_biquadratic_roots(a, b, c):
    roots = []
    d = b ** 2 - 4 * a * c
    print("Дискриминант:", d)
    if d < 0:
        return roots
    if d == 0:
        y = -b / (2 * a)
        if y == 0:
            roots.append(0.0)
        elif y > 0:
            roots.append(math.sqrt(y))
            roots.append(-math.sqrt(y))
    else:
        sqrt_d = math.sqrt(d)
        y1 = (-b + sqrt_d) / (2 * a)
        y2 = (-b - sqrt_d) / (2 * a)
        if y1 == 0:
            roots.append(0.0)
        elif y1 > 0:
            roots.append(math.sqrt(y1))
            roots.append(-math.sqrt(y1))
        if y2 == 0:
            if 0.0 not in roots:
                roots.append(0.0)
        elif y2 > 0:
            roots.append(math.sqrt(y2))
            roots.append(-math.sqrt(y2))
    return roots

def main():
    a = get_coef(1, "Введите коэффициент A: ", True)
    b = get_coef(2, "Введите коэффициент B: ")
    c = get_coef(3, "Введите коэффициент C: ")
    roots = get_biquadratic_roots(a, b, c)
    if len(roots) == 0:
        print("Действительных корней нет.")
    else:
        print("Действительные корни:")
        for root in roots:
            print(root)

if __name__ == "__main__":
    main()
```

```
Введите коэффициент A: 0
Коэффициент A не может быть равен 0
Введите коэффициент A: 1
Введите коэффициент B: -4
Введите коэффициент C: 4
Дискриминант: 0.0
Действительные корни:
1.4142135623730951
-1.4142135623730951
```

```
Введите коэффициент A: 1
Введите коэффициент B: 2
Введите коэффициент C: 3
Дискриминант: -8.0
Действительных корней нет.
```

Рис. 1-4. Процедурный подход

С применением ООП

```
import sys
import math

def get_coef(index, prompt, nonzero=False):
    if index < len(sys.argv):
        value = sys.argv[index]
    else:
        value = None
    while True:
        if value is None:
            value = input(prompt)
        try:
            coef = float(value)
            if nonzero and coef == 0:
                print("Коэффициент A не может быть равен 0")
                value = None
                continue
            return coef
        except ValueError:
            print("Некорректное значение, введите действительное число")
            value = None

class BiquadraticEquation:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def discriminant(self):
        return self.b ** 2 - 4 * self.a * self.c
```

```
def roots(self):
    result = []
    d = self.discriminant()
    if d < 0:
        return result
    if d == 0:
        y = -self.b / (2 * self.a)
        self.add_roots_from_y(y, result)
    else:
        sqrt_d = math.sqrt(d)
        y1 = (-self.b + sqrt_d) / (2 * self.a)
        y2 = (-self.b - sqrt_d) / (2 * self.a)
        self.add_roots_from_y(y1, result)
        self.add_roots_from_y(y2, result)
    return result

def add_roots_from_y(self, y, result):
    if y == 0:
        if 0.0 not in result:
            result.append(0.0)
    elif y > 0:
        result.append(math.sqrt(y))
        result.append(-math.sqrt(y))

def main():
    a = get_coef(1, "Введите коэффициент A: ", True)
    b = get_coef(2, "Введите коэффициент B: ")
    c = get_coef(3, "Введите коэффициент C: ")
    equation = BiquadraticEquation(a, b, c)
    print("Дискриминант:", equation.discriminant())
    roots = equation.roots()
```

```
if not roots:
    print("Действительных корней нет.")
else:
    print("Действительные корни:")
    for root in roots:
        print(root)

if __name__ == "__main__":
    main()
```

```
Введите коэффициент A: 0
Коэффициент A не может быть равен 0
Введите коэффициент A: 1
Введите коэффициент B: -4
Введите коэффициент C: 4
Дискриминант: 0.0
Действительные корни:
1.4142135623730951
-1.4142135623730951
```

```
Введите коэффициент A: 1
Введите коэффициент B: 2
Введите коэффициент C: 3
Дискриминант: -8.0
Действительных корней нет.
```

Рис. 5-9. Объектно-ориентированный подход

Сопоставление с образом

```
import sys
import math

def get_coef(index, prompt, nonzero=False):
    if index < len(sys.argv):
        value = sys.argv[index]
    else:
        value = None
    while True:
        if value is None:
            value = input(prompt)
        try:
            coef = float(value)
            if nonzero and coef == 0:
                print("Коэффициент A не может быть равен 0.")
                value = None
                continue
            return coef
        except ValueError:
            print("Некорректное значение, введите действительное число")
            value = None

def discriminant(a, b, c):
    return b ** 2 - 4 * a * c

def quadratic_roots(a, b, c):
    d = discriminant(a, b, c)
    match d:
        case d if d < 0:
            return ()
        case 0:
            return (-b / (2 * a),)
        case d if d > 0:
            sqrt_d = math.sqrt(d)
            return (
                (-b + sqrt_d) / (2 * a),
                (-b - sqrt_d) / (2 * a)
            )

def roots_from_y(y):
    match y:
        case y if y < 0:
            return ()
        case 0:
            return (0.0,)
        case y if y > 0:
            return (
                math.sqrt(y),
                -math.sqrt(y)
            )

def biquadratic_roots(a, b, c):
    y_roots = quadratic_roots(a, b, c)
    return tuple(
        x
        for y in y_roots
        for x in roots_from_y(y)
    )

def main():
    a = get_coef(1, "Введите коэффициент A: ", True)
    b = get_coef(2, "Введите коэффициент B: ")
    c = get_coef(3, "Введите коэффициент C: ")
    d = discriminant(a, b, c)
    print("Дискриминант:", d)
    roots = biquadratic_roots(a, b, c)
    # Убираем возможные одинаковые корни
    roots = tuple(dict.fromkeys(roots))
    match len(roots):
        case 0:
            print("Действительных корней нет.")
        case 1:
            print("Один действительный корень:", roots[0])
        case _:
            print("Действительные корни:")
            for root in roots:
                print(root)

if __name__ == "__main__":
    main()
```

```
Введите коэффициент A: 1
Введите коэффициент B: 2
Введите коэффициент C: 3
Дискриминант: -8.0
Действительных корней нет.
```

```
Введите коэффициент A: 0
Коэффициент A не может быть равен 0
Введите коэффициент A: 1
Введите коэффициент B: -4
Введите коэффициент C: 4
Дискриминант: 0.0
Действительные корни:
1.4142135623730951
-1.4142135623730951
```

Рис. 10-14. Сопоставление с образом

Вывод: в ходе выполнения лабораторной работы были изучены основные синтаксические конструкции языка Python. Разработана консольная программа для нахождения действительных корней биквадратного уравнения.